Apunte 14 - Introducción al Diseño de APIs

Introducción - Repasando ¿Qué es una API?

Una API (Interfaz de Programación de Aplicaciones) define un contrato que permite que dos aplicaciones se comuniquen entre sí. En el contexto web, las APIs RESTful utilizan HTTP como medio de comunicación para exponer servicios que otras aplicaciones pueden consumir.

Protocolo HTTP

El Protocolo de Transferencia de Hipertexto (HTTP) es ampliamente utilizado en internet, debido a que es el protocolo fundamentalmente empleado por las páginas web. Cuando se origina el protocolo HTTP, su concepción original era la de proporcionar un medio para la transferencia de contenido estático. En otras palabras, estaba diseñado para manejar información que se encontrara almacenada en archivos expuestos a través de servidores web. De esta forma, el protocolo ofrece un mecanismo eficaz para la obtención de tales archivos. Inicialmente, dichos archivos consistían mayormente en documentos de texto, aunque con la introducción del lenguaje HTML (Hypertext Markup Language), estas páginas web comenzaron a adoptar elementos de diseño que los navegadores podían interpretar y presentar de manera coherente y estructurada.

Además de su papel inicial en la entrega de contenido estático almacenado en archivos, el HTTP se adaptó para satisfacer la demanda de consumir datos que no necesariamente estaban almacenados en archivos convencionales. En lugar de limitarse únicamente a archivos físicos, el protocolo se transformó para permitir la obtención de datos generados dinámicamente en tiempo de ejecución. Estos datos podían ser generados al instante por el servidor en respuesta a una solicitud, sin requerir su previo almacenamiento en la infraestructura del servidor.

URI

Los recursos dentro del contexto de la arquitectura web están dotados de identidad, denotada por un identificador único. Este identificador de origen se forma a partir del nombre del servidor y un nombre distintivo del recurso en cuestión. En los sitios web los recursos adoptan diversas formas como archivos HTML, scripts JavaScript, hojas de estilo CSS, imágenes y más. Cada uno de estos elementos dispone de su propia denominación exclusiva. La concatenación del nombre del servidor y el identificador del recurso en forman el Uniform Resource Identifier (URI), que identifica de manera única y precisa el recurso de forma única en internet.

La característica principal es que el protocolo HTTP no establece una imposición de que las URI deban necesariamente hacer referencia o identificar entidades físicamente almacenadas en los servidores. En realidad, las URI pueden representar y señalar una amplia gama de recursos, independientemente de si están o no almacenados. Al recibir una solicitud hacia una URI, el servidor tiene la potestad de determinar la acción a seguir.

En una situación donde el servidor identifica una URI que apunta a un recurso físicamente almacenado, puede responder al cliente con el contenido de ese recurso. Esto es especialmente cierto en el caso de archivos estáticos, como páginas HTML, imágenes o archivos de estilo. Pero también existen escenarios en los cuales el servidor, al interpretar la petición recibida, opta por generar datos en tiempo de ejecución como respuesta.

Por ejemplo, en el caso de necesitar exponer datos provenientes de una base de datos. Suponiendo una base de datos de clientes, donde cada cliente está identificado por una clave primaria única, el servidor puede establecer que las URI con el formato "servidor/clientes/5" no hacen referencia a archivos físicos, sino a los datos almacenados en la base de datos del cliente cuya clave primaria es 5.

De una manera similar, si para referenciar a facturas de ventas, puede el servidor establecer que las uri con formato "servidor/facturas/1234" corresponden con los datos de la factura número 1234, mientras que la uri "servidor/facturas/1234/detalles" identifican al conjunto de ítems de detalle de tal factura y que "servidor/facturas/1234/detalles/2" identifica al segundo ítem de la misma.

Principios REST

El estilo arquitectónico REST se basa en una serie de principios que buscan lograr sistemas distribuidos simples y eficientes. Entre ellos:

- **Identificación de recursos** mediante URIs.
- **Manipulación de recursos** a través de representaciones (como JSON).
- **Uso de métodos HTTP estandarizados** (GET, POST, PUT, DELETE).
- **Comunicación sin estado (stateless)**.
- **Sistema en capas**.

Verbos de HTTP

Los verbos que conforman el conjunto de comandos proporcionados por el protocolo son esenciales en el proceso de interacción entre un software cliente y un servidor. El protocolo HTTP se establece como una serie de comandos que definen las acciones que pueden ser ejecutadas en este contexto. En total, se identifican entre el HTTP original y extensiones aproximadamente 40 comandos, de los cuales sólo se emplearán aquellos necesarios para cada situación específica.

Estos comandos, representados como verbos, consisten en las operaciones que un cliente puede solicitar al servidor. Cada uno de ellos denota una acción precisa y define cómo debe ser tratada por el servidor en el momento de recibir la petición. Cada petición generada por un software cliente estará compuesta por dos elementos: el verbo o nombre de la acción que se desea realizar, y la URI, que designa el recurso sobre el cual se pretende ejecutar dicha acción. El servidor, al recibir una petición, interpreta el verbo HTTP como la operación que debe llevar a cabo sobre el recurso señalado por la URI.

Los comandos más utilizados al desarrollar APIs son los siguientes:

GET

Simboliza la acción de obtener un recurso o su representación. Este comando le comunica al servidor el deseo del cliente de obtener una representación actual del estado del recurso. Para una URI como o "clientes/5", el verbo "GET" transmitirá al servidor la intención de acceder al estado más reciente del cliente número 5.

Como respuesta, el servidor determinará cómo representar ese estado y lo presentará al cliente. Esta representación podría implicar una consulta a la base de datos que almacena información sobre los clientes y posteriormente generar una respuesta que refleje la situación actual del cliente, en algún formato acordado con el cliente, tal como JSON.

POST

El verbo "POST" indica la acción de que un cliente envíe un recurso nuevo al servidor. En el contexto de una aplicación REST, el verbo "POST" se utiliza para la inserción de nuevos datos en una base de datos. Al efectuar una petición "POST", el cliente envía la URI que identifica el recurso que se busca crear. A continuación, se proporcionan los datos correspondientes al nuevo recurso. El servidor, tras recibir esta información, evalúa su validez y en tal caso agrega un nuevo elemento con los datos suministrados. En una base de datos relacional, esto se reflejaría como la inserción de una nueva fila en una tabla. El verbo "POST" actúa como una herramienta para la creación de recursos nuevos y la ampliación de la base de datos.

En el caso de las bases de datos relacionales en las que una tabla posea como clave primaria un dato auto-numérico que se genera a causa de la inserción de una fila, el cliente no puede conocer el valor de dicha clave en el momento de realizar el POST. En ese caso, la URI no indica el identificador definitivo que poseerá el recurso luego su creación. En esos casos es esperable que el servidor informe de la URI definitiva como resultado del comando.

PUT

El verbo "PUT", por su parte, adquiere sentido al modificar un recurso ya existente. Al efectuar una petición con el verbo "PUT", el cliente incluye en la URI el recurso que se busca alterar. Por ejemplo, "clientes/5" apuntaría al cliente número 5. La petición incorpora además todos los datos que constituirían el nuevo estado del recurso. El servidor toma esta información y, tras evaluar su validez, aplica los cambios al recurso que ya está almacenado. Es decir que "PUT" opera como una acción de reemplazo, actualizando un recurso con nuevos datos proporcionados por el cliente.

PATCH

El verbo "PATCH" se asemeja a "PUT", pero difiere en su enfoque. Al realizar una petición "PATCH", el cliente se concentra en modificar únicamente partes específicas de un recurso, en lugar de reemplazarlo en su totalidad. Esta operación permite actualizaciones parciales, lo cual es útil cuando sólo se requiere cambiar ciertos atributos del recurso, por ejemplo, deshabilitar un cliente o modificar el stock de un artículo.

DELETE

Cuando un cliente desea eliminar un recurso empleará una petición con el verbo "DELETE" y la correspondiente URI que identifica el recurso a borrar.

Seguridad e idempotencia

La seguridad y la idempotencia son conceptos importantes en el contexto de los comandos HTTP, que definen aspectos cruciales en la interacción entre clientes y servidores en la web.

Seguridad

Los comandos de HTTP se consideran seguros cuando no modifican el estado de los recursos. Particularmente el verbo GET se considera seguro, por lo tanto la programación del servidor debe evitar que un GET realice modificaciones del estado del recurso solicitado, porque las aplicaciones cliente lo van a ejecutar suponiendo dicho comportamiento.

Idempotencia

La idempotencia se refiere a la característica de ciertos verbos de ser capaces de ser ejecutados múltiples veces sin cambiar el estado del recurso en el servidor luego del primer intento. En otras palabras, realizar la misma operación varias veces no debería tener un efecto diferente al realizarla solo una vez. Esto es fundamental para garantizar la coherencia y la predictibilidad en las interacciones entre clientes y servidores.

Los verbos "GET", "PUT" y "DELETE" son considerados idempotentes. Esto significa que realizar múltiples solicitudes con estos verbos no debería cambiar el estado del recurso en el servidor después de la primera solicitud exitosa. Por ejemplo, un "DELETE" debe borrar un recurso la primera vez que se lo ejecuta, pero otro DELETE con la misma URI no debería tener ningún efecto porque dicho recurso ya fue eliminado la primera vez.

Sin embargo, los verbos "POST" y "PATCH" no son indempotentes, ya que realizar la misma solicitud varias veces podría generar múltiples cambios en el estado del recurso en el servidor. Esto es importante tenerlo en cuenta al diseñar aplicaciones y sistemas que utilizan estos verbos, ya que múltiples solicitudes accidentales o repetidas podrían tener consecuencias no deseadas.

En Resumen

Verbo	**Acción**	**Idempotente**	**Seguro**	**Uso típico**
GET	Obtener recurso	☑	☑	Lectura
POST	Crear recurso	✗	✗	Inserción
PUT	Reemplazar recurso	☑	✗	Actualización total
PATCH	Modificar parcialmente	✗	✗	Actualización parcial
DELETE	Eliminar recurso	☑	✗	Eliminación

Códigos de respuesta

El protocolo HTTP está diseñado de manera que cuando un servidor recibe una petición, debe interpretarla y responder con un código de respuesta que informe al cliente sobre el estado de la solicitud. Esta respuesta consta de dos componentes: las cabeceras y el cuerpo de la respuesta.

Las cabeceras contienen metadatos que describen el contenido y el comportamiento de la respuesta. Estos metadatos pueden incluir información sobre el tipo de contenido, las cookies, la fecha de la respuesta, y otros detalles relevantes. Las cabeceras proporcionan contexto adicional para que el cliente comprenda cómo debe procesar el cuerpo de la respuesta.

El cuerpo de la respuesta contiene los datos reales que el servidor envía al cliente como resultado de la solicitud. Esto podría ser HTML, JSON, imágenes u otros tipos de contenido, dependiendo de la naturaleza de la solicitud y el recurso al que se esté accediendo.

Los códigos de respuesta son números enteros que están estandarizados en el protocolo HTTP. Cada código tiene un significado específico que indica el estado de la respuesta del servidor. Por ejemplo, el código "200 OK" indica que la solicitud se ha procesado correctamente y el recurso solicitado se encuentra disponible. Otros códigos, como "404 Not Found", señalan que el recurso solicitado no se ha encontrado en el servidor.

Estos códigos están diseñados para ser comprensibles tanto por máquinas como por desarrolladores, lo que facilita la identificación y el manejo de diferentes situaciones. Además, algunos códigos pueden ser redefinidos o adaptados por los servidores para ajustarse a situaciones específicas. Por ejemplo, una aplicación web puede definir un código de respuesta personalizado para indicar una situación particular que no está cubierta por los códigos estándar.

☑ Tabla en Markdown con los códigos de respuesta más comunes

Código	Significado	Categoría	Uso común
200	OK	Éxito (2xx)	Operación realizada correctamente
201	Created	Éxito (2xx)	Recurso creado con éxito (POST)
204	No Content	Éxito (2xx)	Operación exitosa sin contenido de respuesta
400	Bad Request	Error del cliente (4xx)	Datos mal formados o inválidos
401	Unauthorized	Error del cliente (4xx)	Falta autenticación
403	Forbidden	Error del cliente (4xx)	Autenticado pero sin permisos
404	Not Found	Error del cliente (4xx)	Recurso no encontrado
422	Unprocessable Entity	Error del cliente (4xx)	Falla de validaciones o reglas de negocio
500	Internal Server Error	Error del servidor (5xx)	Fallo inesperado en el servidor

> 🐱 Para explorar una representación divertida (y muy útil) de los códigos HTTP: Visitá 👉
> https://http.cat
> Vas a encontrar una imagen de gato para cada código. ¡Ideal para recordarlos con una sonrisa!

Cuerpo de las respuestas

Después del código de respuesta en una interacción HTTP, es común encontrar el contenido de la respuesta. Este contenido es opcional, pero suele ser incluido en la mayoría de los casos. El contenido de la respuesta es una parte vital de la comunicación entre el servidor y el cliente, ya que es la información que el servidor envía como resultado de la solicitud realizada.

En el contexto de una solicitud "GET" para obtener el estado de un recurso, por ejemplo, el servidor responderá con un código de respuesta que indica el resultado de la solicitud, como "200 OK" si todo está bien, y luego incluirá el contenido que describe el estado del recurso solicitado. El contenido de la respuesta es un componente de texto, ya que los protocolos de comunicación en la web son mayoritariamente basados en texto.

El tipo de contenido que se envía en la respuesta se define en la cabecera "Content-Type". Esta cabecera indica al cliente el formato en el que se encuentra el contenido de la respuesta, permitiéndole interpretarlo correctamente. Los tipos de contenido comunes incluyen:

- `text/plain`: Este tipo indica que el contenido es texto sin formato.
- `text/html`: Usado cuando se envía contenido en formato HTML.
- `application/json`: Utilizado cuando el contenido está en formato JSON. JSON es ampliamente utilizado para estructurar datos y se utiliza para representar objetos y estructuras en la respuesta.
- `application/xml`: Empleado para contenido en formato XML.

La elección del formato de contenido depende de la naturaleza de los datos y de la forma en que el servidor y el cliente interactúan. JSON se ha convertido en uno de los formatos más populares debido a su facilidad de uso y legibilidad, así como a la facilidad con la que puede ser interpretado tanto por máquinas como por desarrolladores.

Es importante destacar que si el contenido es binario (como una imagen) debe ser codificado en un formato de texto. Un ejemplo común de esto es el uso de codificación Base64 para enviar contenido binario en una respuesta HTTP.

Categorías de las respuestas

Los códigos de respuesta en el protocolo HTTP están estandarizados y siguen una estructura específica para informar sobre el resultado de una acción o solicitud. Estos códigos están compuestos por números enteros de tres dígitos, y cada uno proporciona una categorización y contexto sobre la respuesta del servidor.

Los códigos de respuesta están categorizados según el primer dígito del número:

- Informativos (1xx): Estos códigos indican que la solicitud ha sido recibida y el servidor continúa procesando. Por ejemplo, "100 Continue" significa que el servidor está esperando que el cliente continúe con la solicitud.

- Satisfactorios (2xx): Los códigos que comienzan con "2" indican que la solicitud fue recibida, comprendida y aceptada correctamente. "200 OK" es el más habitual e indica que la solicitud se procesó sin problemas.

- Redirección (3xx): Estos códigos señalan que el cliente necesita realizar una acción adicional para completar la solicitud. Por ejemplo, "301 Moved Permanently" indica que el recurso solicitado ha sido trasladado de manera permanente a otra ubicación.

- Error del cliente (4xx): Los códigos que empiezan con "4" indican que ha habido un error por parte del cliente. "404 Not Found" es un ejemplo común que indica que el recurso solicitado no pudo ser encontrado en el servidor.

- Error del servidor (5xx): Los códigos que comienzan con "5" apuntan a errores que ocurren en el lado del servidor. "500 Internal Server Error" es un ejemplo conocido, indicando que ha ocurrido un error interno en el servidor.

Uso para CRUD

La reutilización de códigos de respuesta en una API REST que implementa operaciones CRUD (Crear, Leer, Actualizar, Eliminar) es una práctica importante para mantener la consistencia en la comunicación entre el servidor y el cliente. Estos códigos de respuesta tienen significados que ayudan a los clientes a entender el estado y el resultado de sus solicitudes. Si bien pueden ser reinterpretados a conveniencia de cada caso es importante aplicarlos de manera coherente. Así, si entre frontend y backend se establece que cierto código de

respuesta va a ser interpretado con un significado diferente al estándar, ese nuevo significado debe ser aplicado consistentemente en todo el sistema.

A continuación se presentan algunos ejemplos de cómo se pueden aplicar los códigos de respuesta en una API REST que implementa operaciones CRUD:

- GET (Leer):
    - Código 200: Se utiliza cuando la solicitud de consulta es exitosa y se devuelve el recurso solicitado.
    - Código 404: Indica que el recurso no se encontró en el servidor.
- POST (Crear):
    - Código 201: Se emplea cuando la solicitud de inserción de un nuevo recurso es exitosa. Junto con este código, la respuesta podría incluir la URI del recurso creado.
    - Código 400: Se usa si la solicitud está mal formada o no se pueden procesar los datos enviados por el cliente.
- PUT / PATCH (Actualizar):
    - Código 200: Puede usarse para indicar que la actualización ha sido exitosa.
    - Código 204: Indica que la actualización fue exitosa y no hay contenido adicional que enviar en la respuesta.
    - Código 404: Si el recurso no se encuentra.
- DELETE (Eliminar):
    - Código 204: Puede utilizarse para indicar que la eliminación ha sido exitosa y no hay contenido adicional que enviar en la respuesta.
    - Código 404: Si el recurso no se encuentra.
- Errores:
    - Código 400: Indica una solicitud mal formada o con datos incorrectos.
    - Código 401: Indica que el cliente no está autorizado para acceder al recurso solicitado.
    - Código 403: Indica que el cliente no tiene permiso para acceder al recurso.
    - Código 404: Indica que el recurso no se encontró.
    - Código 422: Se utiliza cuando los datos enviados por el cliente no cumplen con las reglas de negocio o son inválidos para la acción solicitada.
    - Código 500: Indica un error interno del servidor.

En todos los casos, es importante que el servidor proporcione contenido en la respuesta que explique la situación. En las respuestas exitosas, este contenido puede ser el propio recurso solicitado o una confirmación del éxito de la operación. En las respuestas de error, el contenido puede incluir detalles adicionales sobre el error para ayudar al cliente a comprender qué salió mal.

Estrategias para Definir Endpoints

1. Recursos: Identificar claramente los recursos que la API va a gestionar. Por ejemplo, usuarios, productos, pedidos, etc.

2. Acciones: Definir qué acciones se pueden realizar sobre esos recursos utilizando los métodos HTTP adecuados:
    - GET: Recuperar información sobre un recurso.
    - POST: Crear un nuevo recurso.
    - PUT / PATCH: Actualizar un recurso existente.
    - DELETE: Eliminar un recurso.

Ejemplo de Estructura de Endpoints

Imaginemos una API para gestionar un sistema de comercio electrónico. La estructura de endpoints podría ser la siguiente:

- Usuarios
    - GET /api/v1/users/{id}: Obtener detalles de un usuario específico.
    - POST /api/v1/users: Crear un nuevo usuario.
    - PUT /api/v1/users/{id}: Actualizar un usuario existente.
    - DELETE /api/v1/users/{id}: Eliminar un usuario.
    - GET /api/v1/users: Obtener una lista de usuarios.
- Productos
    - GET /api/v1/products: Obtener una lista de productos.
    - GET /api/v1/products/{id}: Obtener detalles de un producto específico.
    - POST /api/v1/products: Crear un nuevo producto.
    - PUT /api/v1/products/{id}: Actualizar un producto existente.
    - DELETE /api/v1/products/{id}: Eliminar un producto.
- Pedidos
    - GET /api/v1/orders: Obtener una lista de pedidos.
    - GET /api/v1/orders/{id}: Obtener detalles de un pedido específico.
    - POST /api/v1/orders: Crear un nuevo pedido.
    - PUT /api/v1/orders/{id}: Actualizar un pedido existente.
    - DELETE /api/v1/orders/{id}: Eliminar un pedido.

Buenas Prácticas para la Definición de Endpoints

- Claridad y Consistencia: Los endpoints deben ser claros y consistentes en su nomenclatura y estructura. Utiliza nombres descriptivos y sigue convenciones de nombramiento comunes, como el uso de sustantivos plurales para recursos (por ejemplo, /users, /orders).

    1. **Uso de Sustantivos**: Utiliza sustantivos en lugar de verbos en los URIs para representar recursos. Prefiere los plurales para indicar colecciones genéricas.

    ```
    Correcto: https://api.example.com/users
    Incorrecto: https://api.example.com/getUsers
    ```

2. **Uso de Nombres Claros e Intuitivos**: Elige nombres que sean fáciles de entender y representen claramente el recurso o acción. Evita abreviaturas y términos vagos.

```
Correcto: https://api.example.com/users
Incorrecto: https://api.example.com/users/fn
```

3. **Uso de Letras Minúscula**: Utiliza letras minúsculas para los URIs, ya que es una convención aceptada y evita problemas de sensibilidad a mayúsculas.

```
Correcto: https://api.example.com/users
Incorrecto: https://api.example.com/Users
```

4. **Evitar Uso de Caracteres Especiales**: Evita caracteres especiales en los URIs para mantener claridad y evitar problemas técnicos.

```
Correcto: https://api.example.com/users
Incorrecto: https://api.example.com/user%20details
```

5. **Uso de Diagonal o Forward Slash (/)**: Utiliza la diagonal para denotar la jerarquía en los URIs, mostrando la relación entre recursos.

```
Correcto: https://api.example.com/users/1234/first-name
Incorrecto: https://api.example.com/users-1234-first-name
```

6. **Separar Palabras con (-) Guiones o Hyphens**: Utiliza SpinalCase, separa palabras en URIs con guiones para mejorar la legibilidad y el rastreo web.

```
Correcto: https://api.example.com/users/first-name
Incorrecto: https://api.example.com/users/first_name
```

7. **Evitar el Uso de Extensiones de Archivos**: No utilices extensiones de archivos en los URIs, ya que pueden causar problemas y no son necesarias.

```
Correcto: https://api.example.com/users
Incorrecto: https://api.example.com/users.json
```

8. **Uso de camelCase para Parámetros**: Utiliza camelCase para los parámetros en los URIs para distinguirlos claramente.

```
Correcto: https://api.example.com/users/{userId}
Incorrecto: https://api.example.com/users/{userid}
```

9. **Uso de Versionamiento de API**: Implementa el versionamiento en los URIs para gestionar cambios sin romper las APIs existentes.

```
Correcto: https://api.example.com/v1/users
Incorrecto: https://api.example.com/users
```

10. **Consistencia**: Mantén una nomenclatura consistente a lo largo del tiempo y en toda la API para evitar confusión.

```
```plain
Correcto: https://api.example.com/v1/users y
https://api.example.com/v1/orders
Incorrecto: https://api.example.com/v1/users y
https://api.example.com/v2/orders
```
```

- Versionado: Implementa un esquema de versionado en los endpoints para gestionar cambios en la API sin interrumpir a los clientes existentes. Esto se puede hacer mediante prefijos en la URL (por ejemplo, /v1/users).
- Seguridad: Protege los endpoints con mecanismos de autenticación y autorización adecuados. Utiliza HTTPS para cifrar la comunicación y aplica controles de acceso basados en roles para restringir el acceso a las operaciones sensibles.
- Documentación: Documenta los endpoints de manera exhaustiva, incluyendo detalles sobre las operaciones disponibles, los parámetros requeridos, las posibles respuestas y los códigos de error. Herramientas como Swagger pueden ser útiles para generar documentación interactiva de la API.
- Resultados: Elementos claves a tener en cuenta en la obtención de resultados son la Paginación y la poisibilidad de Filtrar o Buscar recursos específicos.
    - Paginar Resultados: En caso de que los recursos sean numerosos, paginar los resultados para evitar respuestas muy grandes que puedan afectar el rendimiento.
    - Filtros y Búsquedas: Permitir el uso de filtros y parámetros de búsqueda en los endpoints para mejorar la flexibilidad y eficiencia en la recuperación de datos.
- Manejo de Errores: Define un esquema claro para el manejo de errores y asegúrate de que los endpoints devuelvan códigos de estado HTTP adecuados y mensajes de error informativos.

Herramientas de diseño y documentación de APIs

A medida que las APIs se vuelven más complejas y utilizadas por múltiples equipos (frontend, backend, integradores externos, etc.), se hace necesario contar con herramientas que permitan **diseñarlas, documentarlas y probarlas de forma estructurada y colaborativa**.

Algunas de las herramientas más utilizadas en la industria son:

🔧 Herramientas de diseño visual y colaborativo

- **Stoplight**: permite diseñar APIs usando OpenAPI con una interfaz visual, soporta mockeo y validación automática.
- **Postman**: además de ser una herramienta de pruebas, permite documentar colecciones y flujos de interacción con APIs.
- **Insomnia**: muy útil para probar y visualizar APIs REST, con soporte para Swagger y GraphQL.
- **Apicurio**: enfocado en el diseño colaborativo de contratos OpenAPI/Swagger.

📖 Especificación de contratos y documentación

- **OpenAPI Specification (OAS)**: estándar para describir APIs RESTful. Define los endpoints, parámetros, respuestas, etc., de forma legible tanto por humanos como por máquinas.
- **RAML**: alternativa a OpenAPI para describir APIs, con foco en reutilización y modularidad.
- **AsyncAPI**: diseñada para APIs orientadas a eventos (como Kafka o WebSockets), complementaria a OpenAPI.

🌟 Introducción a Swagger

Swagger es una herramienta que se apoya en la especificación OpenAPI y permite:

- Describir formalmente la API mediante anotaciones en el código o archivos YAML/JSON.
- Generar documentación interactiva navegable desde el navegador.
- Probar en tiempo real los endpoints desde dicha documentación (Swagger UI).
- Generar clientes o servidores automáticamente a partir del contrato.

> ⚡ En los siguientes apartados profundizaremos en cómo aplicar Swagger en nuestros proyectos y cómo integrarlo en un backend construido con Spring Boot.

Documentación de APIs

La documentación de una API es un componente crítico para el éxito de cualquier proyecto con arquitectura de microservicios que dependa de la interacción entre sus componentes o con diferentes sistemas o aplicaciones. Dado que las APIs actúan como un puente entre distintos servicios, es fundamental que su funcionamiento esté claramente explicado y accesible para los desarrolladores que las utilizan. Una API sin documentación es esencialmente un "código oscuro", lo que significa que incluso si es funcional, su utilidad se ve severamente limitada porque nadie, aparte de los creadores originales, puede entender cómo interactuar con ella correctamente.

La documentación de una API debería proporcionar una guía completa y detallada sobre cada uno de los endpoints disponibles, incluyendo:

1. **Descripción de Endpoints**: Cada endpoint de la API debe estar claramente descrito, explicando su propósito y cuándo debe ser utilizado. Esto permite a los desarrolladores entender qué recursos o

acciones están disponibles y cómo pueden ser accedidos.

2. **Parámetros de Request**: Es esencial que la documentación especifique todos los parámetros que pueden o deben ser enviados en una solicitud (request), incluyendo tipos de datos, si son obligatorios o opcionales, y ejemplos prácticos de uso. Esto ayuda a prevenir errores comunes y garantiza que las solicitudes estén bien formadas.

3. **Códigos de Estado y Respuestas (Response)**: Además de describir los parámetros de entrada, la documentación debe detallar las posibles respuestas que se recibirán, incluyendo los códigos de estado HTTP y los cuerpos de respuesta (response bodies). Esto es crucial para que los desarrolladores puedan manejar correctamente los resultados de sus solicitudes, ya sea éxito, error o alguna condición intermedia.

4. **Ejemplos de Uso**: Incluir ejemplos claros y prácticos de cómo interactuar con la API es una de las formas más efectivas de hacer que la documentación sea útil. Estos ejemplos no solo muestran cómo se forman las solicitudes y respuestas, sino que también ofrecen contexto sobre situaciones comunes de uso.

Una buena documentación no solo facilita el proceso de integración, sino que también mejora la mantenibilidad de la API a lo largo del tiempo. Al proporcionar una referencia clara y accesible, se reduce el tiempo necesario para que nuevos desarrolladores comprendan y comiencen a trabajar con la API. Además, una documentación bien estructurada ayuda a minimizar los errores y malentendidos, lo que resulta en un desarrollo más eficiente y en una mejor experiencia para todos los usuarios de la API.

Swagger



Swagger es una especificación de código abierto y una suite de herramientas que se utilizan para describir, construir y documentar servicios web basados en HTTP. La especificación de Swagger, también conocida como **OpenAPI Specification**, utiliza un formato JSON o YAML para describir la API. Esta descripción incluye información como la versión de la API, los endpoints disponibles, los métodos HTTP permitidos (como GET, POST, PUT, DELETE), los parámetros necesarios y opcionales, los códigos de respuesta esperados y los esquemas de datos utilizados.

Una de las principales ventajas de Swagger es que permite generar automáticamente una documentación interactiva de la API a partir de la especificación. Esta documentación incluye detalles sobre cómo llamar a cada endpoint, qué parámetros se deben proporcionar y qué respuestas se pueden esperar. Además, Swagger

también puede generar código cliente y servidor en varios lenguajes de programación a partir de la especificación, lo que facilita la implementación y el consumo de la API.

Otra característica importante de Swagger es su capacidad para realizar pruebas y validaciones de la API. Puedes enviar solicitudes de prueba a través de la interfaz de Swagger y verificar si las respuestas son las esperadas. Esto ayuda a detectar posibles problemas antes de implementar la API en producción.

Swagger incluye varias herramientas clave:

1. **Swagger Editor**: Una interfaz web intuitiva para escribir y editar definiciones de API en el formato OpenAPI.
2. **Swagger UI**: Una interfaz de usuario interactiva que genera y muestra la documentación de la API en tiempo real, permitiendo a los usuarios interactuar con la API directamente desde el navegador.
3. **Swagger Codegen**: Herramienta para generar código cliente y servidor en varios lenguajes de programación a partir de una definición de API.
4. **Swagger Hub**: Una plataforma en la nube que permite la colaboración en el diseño y documentación de APIs, combinando el editor de Swagger y el Swagger UI en un entorno colaborativo.

Gracias a su enfoque en la claridad, la estandarización y la facilidad de uso, Swagger se ha convertido en una herramienta fundamental para equipos de desarrollo que buscan asegurar que sus APIs sean consistentes, bien documentadas y fáciles de integrar por terceros.

Swagger Editor y Open API

E## Swagger Editor

Swagger Editor permite diseñar, describir y documentar una API en línea. Es compatible con múltiples especificaciones de API y formatos de serialización. El editor de Swagger ofrece una manera fácil de comenzar con la especificación OpenAPI (anteriormente conocida como Swagger). Solo es necesario conocer cómo la OpenAPI permite describir las funcionalidades a desarrollar y escribirlas en un formato YML o JSON. La herramienta automáticamente genera visualmente la especificación de la API permitiendo además probarla desde el mismo sitio.

Pasos para documentar con **Swagger Editor**

1. Acceder a https://editor.swagger.io/ en el navegador web.

2. Es posible comenzar con una especificación en blanco o cargar una especificación existente en formato JSON o YAML. Se puede cargar un archivo local o proporcionar una URL que apunte a la ubicación del archivo de especificación.

3. Una vez que se haya cargado o creado la especificación, el editor Swagger se abrirá en la interfaz principal.

4. En el editor Swagger, se puede comenzar a definir la API REST. A continuación, se muestra una descripción general de las acciones comunes que puedes realizar:

    - Definir información general de la API: En la sección "info" se puede proporcionar detalles como el título, la versión, la descripción y la información de contacto de la API.

- Agregar y describir endpoints: En la sección "paths", se pueden agregar los endpoints y describir cada uno de ellos. Se puede especificar el método HTTP (GET, POST, PUT, DELETE, etc.), los parámetros, las respuestas esperadas y cualquier otra información relevante.
- Definir modelos de datos: En la sección "components/schemas", se pueden definir los modelos de datos utilizados en la API. Se puede especificar las propiedades, los tipos de datos, las restricciones y las relaciones entre los modelos.
- Personalizar las configuraciones de Swagger: Se puede ajustar las configuraciones generales de Swagger en la sección "components".

5. A medida que se haya definiendo la API en el editor Swagger, se podrá ver una representación visual y estructurada de la especificación en la parte derecha de la pantalla. Esta vista previa permite verificar cómo se está construyendo la API en tiempo real.
6. Mientras se trabaja en la especificación, el editor Swagger ofrece funciones útiles como la validación en tiempo real para asegurarse de que sigas el formato correcto y no haya errores.
7. Una vez completada la documentación, se pueden utilizar las opciones de exportación para guardar la especificación en formato JSON o YAML en la computadora local. Además de esta opción, el editor Swagger también ofrece otras funcionalidades como la **generación de código**, donde se puede elegir generar código cliente o servidor en diferentes lenguajes de programación a partir de la especificación definida.

Para mayor detalle se este video con un ejemplo de uso de la herramienta.

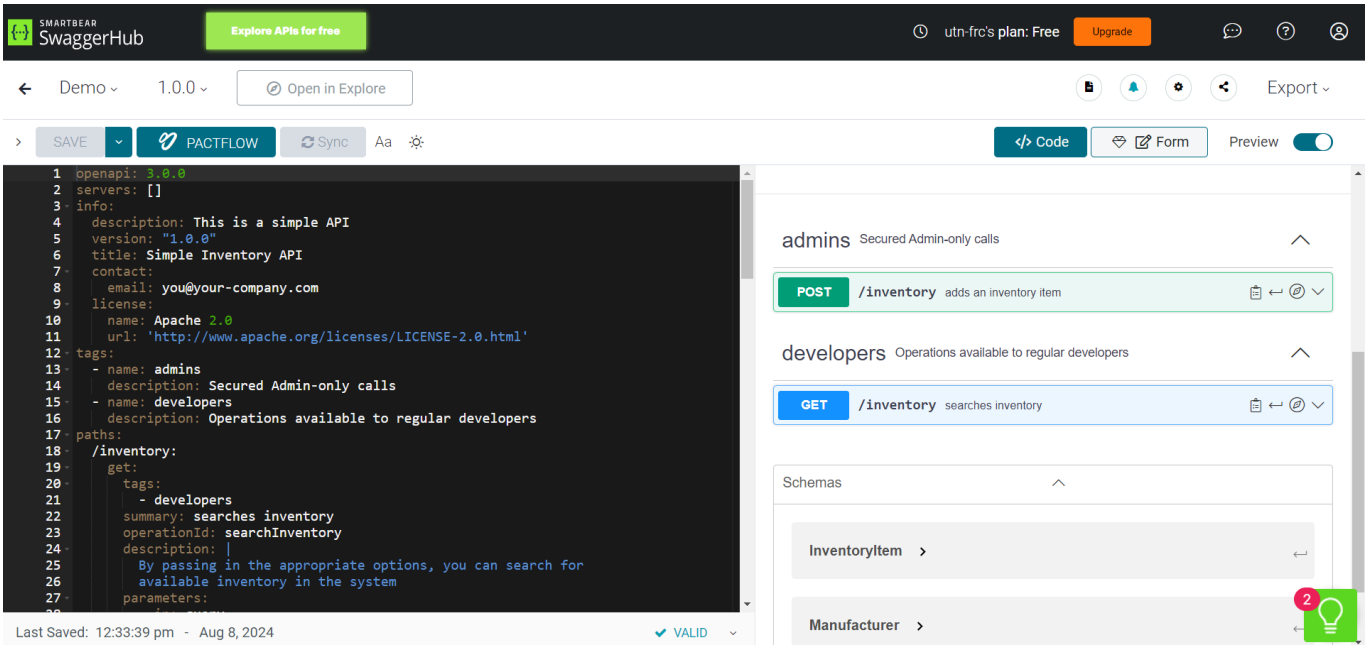


Figura 5: SwaggerUI designer

Introducción a OpenAPI

OpenAPI (anteriormente conocida como Swagger Specification) es una especificación que define una interfaz estándar para describir APIs RESTful. A través de un archivo en formato YAML o JSON, los desarrolladores pueden especificar los endpoints, métodos HTTP, parámetros, respuestas, y otros detalles clave de la API. Este

archivo sirve como la única fuente de verdad, permitiendo generar documentación automática, código cliente, servidores mock, y más.

```yaml
openapi: 3.0.0
servers: []
info:
  description: This is a simple API
  version: "1.0.0"
  title: Simple Inventory API
  contact:
    email: you@your-company.com
  license:
    name: Apache 2.0
    url: 'http://www.apache.org/licenses/LICENSE-2.0.html'
tags:
  - name: admins
    description: Secured Admin-only calls
  - name: developers
    description: Operations available to regular developers
paths:
  /inventory:
    get:
      tags:
        - developers
      summary: searches inventory
      operationId: searchInventory
      description: |
        By passing in the appropriate options, you can search for
        available inventory in the system
      parameters:
```

> Figura 6: Cógido Open Api

Un documento OpenAPI sigue una estructura jerárquica que organiza la información de la API en secciones bien definidas. A continuación, se describen los elementos principales que componen un archivo OpenAPI:

1. **openapi**: Define la versión de la especificación OpenAPI que se está utilizando (por ejemplo, `3.0.0`).
2. **info**: Proporciona metadatos sobre la API, incluyendo:
    - **title**: El título de la API.
    - **description**: Una descripción breve de la API.
    - **version**: La versión de la API.
3. **servers**: Una lista de URLs de servidores donde la API está disponible. Puede incluir diferentes entornos como desarrollo, prueba, y producción.
4. **paths**: Especifica los endpoints de la API y los métodos HTTP que pueden ser utilizados. Cada endpoint (o ruta) contiene:
    - **summary** y **description**: Información adicional sobre el propósito del endpoint.
    - **operationId**: Un identificador único para la operación.
    - **parameters**: Una lista de parámetros que la operación acepta, que pueden ser query, path, header, o cookie parameters.
    - **responses**: Las posibles respuestas que puede devolver la API, con códigos de estado HTTP y la estructura del cuerpo de la respuesta.

5. **components**: Define esquemas reutilizables, parámetros, respuestas y otros objetos que pueden ser referenciados en múltiples partes de la especificación. Dentro de **components**, podemos encontrar:
    - **schemas**: Definiciones de los modelos de datos que la API maneja, utilizando esquemas JSON.
    - **securitySchemes**: Esquemas de seguridad que detallan cómo la API está protegida (por ejemplo, autenticación mediante API keys, OAuth2, etc.).
6. **security**: Define los requisitos de seguridad globales para la API o para endpoints específicos, especificando qué esquemas de seguridad se aplican.
7. **tags**: Permite agrupar operaciones de la API bajo etiquetas comunes, facilitando la organización y navegación de la documentación.
8. **externalDocs**: Un enlace a documentación externa relacionada con la API, si es necesario.

Funcionamiento de Swagger Editor

El Swagger Editor es una aplicación web que permite a los desarrolladores escribir definiciones OpenAPI de manera interactiva. Su interfaz proporciona una vista en dos paneles: un editor de texto donde se escribe el archivo OpenAPI y una vista previa que muestra cómo se verá la documentación generada.

1. **Escritura y Validación en Tiempo Real**: A medida que escribes la definición de la API en el editor, Swagger Editor valida el contenido en tiempo real, asegurando que se ajusta a la especificación OpenAPI. Los errores y advertencias se muestran de inmediato, lo que facilita la corrección rápida.
2. **Vista Previa de la Documentación**: En el panel derecho, se genera automáticamente una vista previa interactiva de la documentación basada en la definición que se está escribiendo. Esto permite ver cómo los usuarios de la API verán y entenderán los endpoints y las operaciones disponibles.
3. **Generación de Esqueleto de Código**: A partir de la definición OpenAPI, es posible generar esqueleto de código para el cliente y el servidor en varios lenguajes de programación utilizando herramientas como Swagger Codegen. Esto acelera el desarrollo, asegurando que la implementación sea consistente con la especificación.

Alternativas de Uso

El Swagger Editor, junto con OpenAPI, puede ser utilizado tanto para la **documentación** como para el **diseño** de una API:

1. **Documentación de APIs Existentes**: Si ya tienes una API en funcionamiento, puedes utilizar Swagger Editor para crear una especificación OpenAPI que documente todos los endpoints, parámetros, y respuestas. Esto no solo mejora la comunicación entre desarrolladores, sino que también permite la generación automática de documentación interactiva mediante Swagger UI.
2. **Diseño de Nuevas APIs**: Antes de escribir una sola línea de código, puedes utilizar el Swagger Editor para diseñar una nueva API. Definir la API primero en OpenAPI permite iterar sobre el diseño, recibir retroalimentación, y asegurarte de que todos los requisitos están cubiertos antes de comenzar la implementación. Este enfoque "API-First" también facilita la creación de mocks de la API para pruebas tempranas.

Más documentación de Swagger

- Artículo acerca de Buenas prácticas en el Diseño de APIs
- Documentación de Open API 3